



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»

КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА» (ИБМ-3)

Рубежный контроль №1

По дисциплине:

«Парадигмы и конструкции языков программирования»

Студент ИБМЗ- 34Б

(Подпись, дата)

П. Я. Головастикова

Руководитель

(Подпись, дата)

Ю. Е. Гапанюк

2025 г.

Код программы:

```
import unittest
from operator import itemgetter

class Microprocessor:
    def __init__(self, id, model, speed, com_id):
        self.id = id
        self.model = model
        self.speed = speed
        self.com_id = com_id

class Computer:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class ComputerMicroprocessor:
    def __init__(self, com_id, mic_id):
        self.com_id = com_id
        self.mic_id = mic_id

class ComputerService:
    def __init__(self, computers, microprocessors, comp_microproc):
        self.computers = computers
        self.microprocessors = microprocessors
        self.comp_microproc = comp_microproc

    def get_one_to_many(self):
        return [
            (m.model, m.speed, c.name)
            for m in self.microprocessors
            for c in self.computers
            if m.com_id == c.id
        ]

    def get_many_to_many(self):
        many_to_many_temp = [
            (c.name, cm.com_id, cm.mic_id)
            for c in self.computers
            for cm in self.comp_microproc
            if c.id == cm.com_id
        ]

        return [
            (m.model, m.speed, com_name)
            for com_name, com_id, mic_id in many_to_many_temp
            for m in self.microprocessors if m.id == mic_id
        ]

    def task_a1(self):
        one_to_many = self.get_one_to_many()
        return sorted(one_to_many, key=itemgetter(2))

    def task_a2(self):
        one_to_many = self.get_one_to_many()
        res_2_unsorted = []

        for c in self.computers:
            c_micr = list(filter(lambda i: i[2] == c.name, one_to_many))
```

```

        if len(c_micr) > 0:
            c_speed = [speed for _, speed, _ in c_micr]
            c_sum_speed = sum(c_speed)
            res_2_unsorted.append((c.name, c_sum_speed))

    return sorted(res_2_unsorted, key=itemgetter(1), reverse=True)

def task_a3(self):
    many_to_many = self.get_many_to_many()
    res_3 = {}

    for c in self.computers:
        if 'отдел' in c.name:
            c_micr = list(filter(lambda i: i[2] == c.name, many_to_many))
            c_micr_model = [x for x, _, _ in c_micr]
            res_3[c.name] = c_micr_model

    return res_3

class TestComputerService(unittest.TestCase):

    def setUp(self):
        self.computers = [
            Computer(1, "Игровой компьютер отдел разработки"),
            Computer(2, "Серверный компьютер"),
            Computer(3, "Офисный компьютер отдел продаж"),
            Computer(4, "Рабочая станция отдел дизайна"),
            Computer(5, "Ноутбук")
        ]

        self.microprocessors = [
            Microprocessor(1, "Intel Core i7", 3.8, 1),
            Microprocessor(2, "AMD Ryzen 9", 4.2, 1),
            Microprocessor(3, "Intel Xeon", 2.5, 2),
            Microprocessor(4, "Intel Core i5", 3.2, 3),
            Microprocessor(5, "AMD Ryzen 7", 3.6, 4),
            Microprocessor(6, "Intel Core i3", 2.9, 5)
        ]

        self.comp_microproc = [
            ComputerMicroprocessor(1, 1),
            ComputerMicroprocessor(2, 1),
            ComputerMicroprocessor(3, 2),
            ComputerMicroprocessor(4, 3),
            ComputerMicroprocessor(5, 4),
            ComputerMicroprocessor(6, 5),
            ComputerMicroprocessor(1, 3),
            ComputerMicroprocessor(2, 4)
        ]

        self.service = ComputerService(
            self.computers,
            self.microprocessors,
            self.comp_microproc
        )

    def test_task_a1_sorted_by_computer_name(self):
        result = self.service.task_a1()
        computer_names = [item[2] for item in result]
        sorted_computer_names = sorted(computer_names)

        self.assertEqual(computer_names, sorted_computer_names)
        for item in result:

```

```

        self.assertEqual(len(item), 3)
        self.assertIsInstance(item[0], str)
        self.assertIsInstance(item[1], float)
        self.assertIsInstance(item[2], str)

    def test_task_a2_sum_speeds_correctness(self):
        result = self.service.task_a2()
        speeds = [item[1] for item in result]
        sorted_speeds_desc = sorted(speeds, reverse=True)
        self.assertEqual(speeds, sorted_speeds_desc)
        result_dict = dict(result)
        expected_speed_com1 = 3.8 + 4.2
        self.assertAlmostEqual(result_dict["Игровой компьютер отдел
разработки"], expected_speed_com1)

        for computer_name, total_speed in result:
            self.assertIsInstance(computer_name, str)
            self.assertIsInstance(total_speed, float)
            self.assertGreater(total_speed, 0)

    def test_task_a3_department_computers(self):
        result = self.service.task_a3()

        for computer_name in result.keys():
            self.assertIn('отдел', computer_name)

        expected_department_computers = [
            "Игровой компьютер отдел разработки",
            "Офисный компьютер отдел продаж",
            "Рабочая станция отдел дизайна"
        ]

        for expected_computer in expected_department_computers:
            self.assertIn(expected_computer, result)

        self.assertNotIn("Серверный компьютер", result)
        self.assertNotIn("Ноутбук", result)

        for computer_name, microprocessor_models in result.items():
            self.assertIsInstance(microprocessor_models, list)
            for model in microprocessor_models:
                self.assertIsInstance(model, str)

if __name__ == '__main__':
    unittest.main()

```

Результат выполнения:

```

collecting ... collected 3 items
|
PK2ТТД.py::TestComputerService::test_task_a1_sorted_by_computer_name PASSED [ 33%]
PK2ТТД.py::TestComputerService::test_task_a2_sum_speeds_correctness PASSED [ 66%]
PK2ТТД.py::TestComputerService::test_task_a3_department_computers PASSED [100%]

```

В рефакторинге программы я сделала добавление новых классов, отвечающих за выполнение различных связей и отдельных заданий для наших основных классов.

В качестве TTD тестов я провела: первый тест – проверка сортировки по имени компьютера (проверка по третьему элементу кортежа). Второй тест был направлен на проверку корректности подсчета суммарной скорости компьютеров. Третий тест – это проверка фильтрации компьютеров со словом «отдел» в названии.